

Eustace the Pirate Monk — Specification

CS 2114 Project 1, Deliverable 2 — Group 87 (Marcos Salas, Aidan McIlvenni)

Java 8, default package, nothing beyond `java.util` and `student.jar`. Numbers marked *default* live in named constants; Marcos tunes them in week 2, and section 6's expected values are recomputed when they change. Attached: `docs/class-diagram.png` (full UML) and `docs/channel-map.png` (the map).

1. Class design

Twelve classes and one interface, each with one job:

1. **Game**: runs the read–dispatch–print loop, computes the current mode, and is the only class that prints. Holds `main`.
2. **CommandParser**: turns one raw line into a **Command**, holds the table of verbs and their usage lines, and owns every typing-slip rule (`DESIGN.md §4` cases 1–6, 8–11, 25, 27).
3. **Command**: an immutable value, a verb word and its argument string.
4. **ChannelMap**: builds the fixed 18-stop map and finds a stop from the name typed.
5. **Location**: one stop, with its key, name, kind, encounter chance and the stops one move away.
6. **Port**: a **Location** where the ship moors, with its nation and prices.
7. **Ship** (abstract): what both sides share, hull, cannons, armour, gold, rum and a **Crew**, plus the rules for dealing and taking damage.
8. **PlayerShip**: where the player is, notoriety, ports visited, and every purchase.
9. **EnemyShip**: a ship the player meets, built from an **EnemyType** and the player's notoriety.
10. **EnemyType**: the base numbers for one kind of enemy, with three shared instances, `MERCHANT`, `PIRATE` and `COAST_GUARD`, as static final constants.
11. **Crew**: head count, morale and greed, and the rules that move them.
12. **Encounter**: one meeting with an enemy, from the roll that creates it through the fight or the flight to the plunder.
13. **Describable** (interface): `String describe()`, implemented by **Location**, **Ship** and **Encounter**, so **Game** prints any of them the same way.

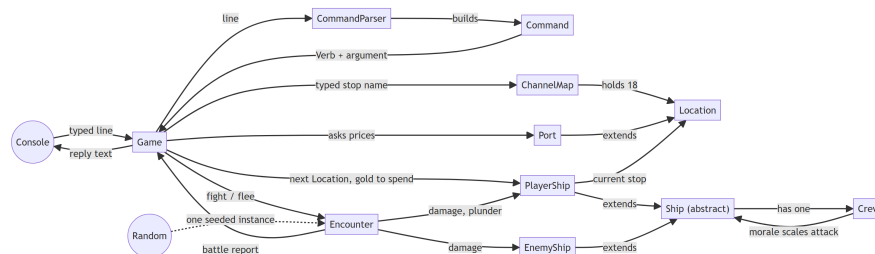
Fixed vocabularies are `String` constants: modes `Game.PORT`, `SEA`, `ENCOUNTER`, `OVER`; kinds `Location.PORT`, `SEA_LANE`, `HIGH_SEAS`; nations "English", "French", "Pirate"; and the verbs `look`, `status`, `sail`, `repair`, `hire`, `buy`, `bonus`, `fight`, `flee`, `retire`, `help`, `quit`.

No class both parses text and changes ship values, and no class both holds game state and prints it.

Why this split: both sides fight by the same rules, so **Encounter** calls `attackStrength()` and `takeDamage()` on a **Ship** without knowing which side it holds. **Crew** is separate because morale scales attack for both sides, and a crew that mutinies ends the run for the player or hands the player the win when it is the enemy's. Cannons and armour are levels 1 to 5, so `buy` takes three fixed words and no port runs out of stock; every port charges the same prices, with `Sark` halving rum. The mode is never stored: `mode()` returns `OVER` after the run ends, `ENCOUNTER` while `encounter != null`, `PORT` at a port, and `SEA` otherwise.

The map: seven ports (three English, three French, and `Sark`, the pirate home), eight sea lanes each joining two ports, and three high-seas stretches. One `sail` moves one stop, so `Dover` to `Boulogne` is `sail dover boulogne` then `sail boulogne`; `find` hyphenates the words typed and, failing that, tries the first word alone.

2. System diagram



Each arrow points the way the labelled thing travels. One turn: **Game** reads a line, **CommandParser** returns a **Command**, **Game** checks the verb against `mode()`, calls the object that owns that verb, and prints what comes back.

3. Data & state

Where a field lists bounds, such as hull from 0 to `maxHull` or morale from 0 to 100, the methods that change it clamp to those bounds, so no caller can drive it outside.

- **Game**: `Scanner in`; final `ChannelMap map` and `PlayerShip player`; `Encounter encounter`, null unless a fight is on; final `Random random`, the only one in the program; boolean `running`; static final `int GOLD_TARGET = 1000 default`.

- **CommandParser**: final `HashMap<String, String>` `usages`, verb word to usage line, filled in the constructor with the twelve verbs, so one lookup answers both "is this a verb" and "what is its usage". `Command`: final `String` `verb`, the first word in lower case or "" for a blank line; final `String` `argument`, lower case, single-spaced, with quotes and ., ! ? stripped from each word's ends, and "" when absent.
- **ChannelMap**: final `HashMap<String, Location>` `stops`, 18 entries keyed by stop key and fixed once `standard()` returns; `Port` `home`. Every sail looks up one typed key.
- **Location**: final `String` `key`, `name` and `kind`; final `int` `encounterPercent`, 0 for a port, 35 for a sea lane and 60 for the high seas *default*; `ArrayList<Location>` `neighbours`, 2 to 4 entries, where if A lists B then B lists A, and a stop never lists itself or a duplicate. `getNeighbours()` hands back the list wrapped in `Collections.unmodifiableList`, so `Game` can loop over the neighbours while add on what it got throws `UnsupportedOperationException` and only `connect` makes links. The list keeps the order look prints.
- **Port** adds final `String` `nation` and the *default* prices `REPAIR_PRICE` = 2, `HIRE_PRICE` = 15, `RUM_PRICE` = 4 and `UPGRADE_PRICE` = 100 gold per unit, with level `n` costing `UPGRADE_PRICE` × `n`.
- **Ship**: final `String` `name`; `int` `hull`, 0 to `maxHull`; final `int` `maxHull`; `int` `cannons`, 1 to 5; `int` `armour`, 0 to 5; `int` `gold` and `int` `rum`, never negative; final `Crew` `crew`. Subclasses change hull, cannons and armour only through protected setters.
- **PlayerShip** adds `Location` `location`, never null; `int` `notoriety`; `int` `portsVisited`; static final `int` `MAX_LEVEL` = 5. It starts *default* at Sark with hull 100, crew 20 at morale 70, cannons 1, armour 1, gold 200 and rum 30.
- **EnemyShip** adds final `EnemyType` `type`. Its constructor takes the type's base numbers and adds an amount set by the player's notoriety: 2 hull per point, 1 crew per 2 points (capped at 40), 1 cannon and 1 armour per 10 points (each capped at 5), and 5 gold per point. Rum starts equal to the crew count and morale comes from the type, so at notoriety 10 a merchant has hull 60, crew 13, cannons 2, armour 1 and gold 200.
- **EnemyType**: final `String` `name`, and final `int` `hull`, `crew`, `cannons`, `armour`, `gold`, `morale` and `gain`. The three *default* constants, in that order: `MERCHANT` 40, 8, 1, 0, 150, 30, 1; `PIRATE` 60, 15, 2, 1, 100, 60, 2; `COAST_GUARD` 80, 20, 3, 2, 50, 80, 3.
- **Crew**: `int` `count`, 0 to `MAX_COUNT` = 40; `int` `morale` and `int` `greed`, both 0 to 100, where greed starts at 0 and only the player's crew ever gains it. *Default* steps: each man lost costs 5 morale, each bottle the crew wanted and did not get costs `MORALE_PER_DRY_BOTTLE` = 5, a win pays 10 back, a flee costs 15, and every 20 gold of plunder adds 1 greed. A bonus of gold to a crew of `c` men is `g` ÷ `c` gold a head: greed falls by that number and morale rises by half of it.
- **Encounter**: final `PlayerShip` `player`, `EnemyShip` `enemy` and `Random` `random`; `FLEE_DAMAGE` = 10 and `ROLL_RANGE` = 6 *default*.

How the numbers move:

1. `moraleFactor()` is $0.5 + \text{morale} \div 200.0$, running from 0.5 at morale 0 to 1.0 at morale 100. `attackStrength()` multiplies that factor by 5 points per cannon plus 1 point per two crew, throws away the fraction and never gives less than 1. A fresh player ship has 1 cannon and 20 crew at morale 70, so $(5 + 10) \times 0.85 = 12.75$, which floors to 12.
2. `takeDamage(raw)` works out what gets through the armour, $\text{raw} - 2 \times \text{armour}$, at least 1 when raw is above 0. The hull drops by that number and the crew loses one man per 5 points of it, rounded down: 12 raw against armour 1 gets 10 through, so hull drops 10 and 2 men die.
3. `isDefeated()` is true at hull 0, at crew 0, or when `crew.mutinied()` is true, which happens at morale 0 or greed 100. The player's crew mutinying ends the run; an enemy crew mutinying stops its fire, printed as a surrender.
4. A fight round: the player fires `attackStrength()` + `random.nextInt(ROLL_RANGE)`; if the enemy still stands it fires back the same way, and rounds repeat until one is defeated. A win moves the enemy's gold and rum across, adds the type's gain to notoriety, and calls `crew.onWin(gold)`.
5. Each sail into a sea stop: the crew wants one bottle per ten men, a part-full ten rounding up, so 20 men want 2 and 21 want 3. It drinks what the hold has, and every bottle it wanted and did not get costs `MORALE_PER_DRY_BOTTLE`: 20 men with 1 bottle aboard drink it and lose 5 morale, and with an empty hold they lose 10. Then `roll` draws `nextInt(100)` against the stop's chance, and a second draw picks the type: in a sea lane below 70 is a merchant, else the coast guard; on the high seas below 50 is a pirate, else a merchant.
6. A purchase buys the smallest of three numbers: the amount typed, how many units the gold covers ($\text{gold} \div \text{price}$, rounded down), and the room left before hull, crew or a level reaches its cap. It then charges that many units times the price, which the gold always covers.

4. Method signatures

"Refuses" means the method returns 0 or false and changes nothing, and `dispatch` turns that into the section 5 message. `IAE` means it throws `IllegalArgumentException`.

`Game:Game(Scanner in, ChannelMap map, Random random); static void main(String[] args)` builds a game on `System.in`, `ChannelMap.standard()` and new `Random()` and runs it; `void run()` prints the opening scene and loops read, parse, dispatch, print until `mode()` is `OVER`; `String dispatch(Command c)` applies one command in the current mode and returns the text to print; `String mode(); PlayerShip getPlayer(); Encounter getEncounter();`

CommandParser: CommandParser() fills the verb table; Command parse(String line) normalises the line and returns a Command holding the first word and the rest, or "" for a blank line; boolean isVerb(String word); String usageOf(String verb) gives the usage line or null; String allUsages() joins every usage line for help; static int parseCount(String text) returns an int of 0 or more, or throws NumberFormatException for anything else.

Command: Command(String verb, String argument), IAE on a null verb; String getVerb(); String getArgument(); String getWord(int i) gives the i-th argument word or ""; String toString() gives "sail dover".

ChannelMap: static ChannelMap standard() builds the 18 stops and 25 links; void add(Location stop), IAE on a duplicate key; void connect(String keyA, String keyB) links both ways, IAE on an unknown key; Location find(String name) gives the stop or null; Port getHome().

Location implements Describable: Location(String key, String name, String kind, int encounterPercent); void connect(Location other) adds each to the other once, IAE on itself; boolean isNextTo(Location other); List<Location> getNeighbours(); getters; String describe() gives name, kind and neighbours. Port: Port(String key, String name, String nation) passes kind PORT and chance 0 upward; String getNation(); int rumPrice(), halved at Sark; int upgradePrice(int nextLevel); describe() adds the prices.

Ship implements Describable: Ship(String name, int maxHull, int cannons, int armour, Crew crew, int gold, int rum); int attackStrength(); int takeDamage(int raw) returns hull lost; boolean isDefeated(); void addGold(int); void spendGold(int), IAE beyond what is held; void addRum(int); int takeRum(int) returns bottles taken, capped at the hold; Crew getCrew(); a getter per field; abstract String describe().

PlayerShip: PlayerShip(String name, Port home); void moveTo(Location d), IAE unless d is a neighbour, counting a port arrival; int repair(int points, int pricePerPoint), int hire(int men, int pricePerMan) and int buyRum(int bottles, int pricePerBottle) each buy by the rule in section 3 and return the units bought; boolean upgradeCannons(int price) and boolean upgradeArmour(int price) refuse at MAX_LEVEL or without the gold; boolean payBonus(int gold) refuses beyond what is held, else hands the gold to the crew; void addNotoriety(int); getters; describe() is the status report.

EnemyShip: EnemyShip(EnemyType type, int notoriety), IAE on null or a negative; EnemyType getType(); describe() is what the lookout sees. EnemyType: a constructor taking the eight values in section 3 and a getter per field; the three constants are its only instances.

Crew: Crew(int count, int morale) caps both; int lose(int men) and int hire(int men) return the number removed or added after capping; double moraleFactor(); int drink(int bottlesAvailable) returns bottles drunk and charges morale for the shortfall; void onWin(int plunder); void onFlee(); void receiveBonus(int gold); boolean mutinied(); getters.

Encounter implements Describable: Encounter(PlayerShip player, EnemyShip enemy, Random random); static Encounter roll(Location where, PlayerShip player, Random random) gives a new encounter or null; String fight() resolves the whole battle and returns the narration; String flee() applies FLEE_DAMAGE and onFlee(); EnemyShip getEnemy(); String describe().

5. Where validation lives

Numbers are DESIGN.md §4 cases; every response leaves all fields unchanged unless the line says otherwise. dispatch checks the mode first: fight and flee need ENCOUNTER; repair, hire, buy and bonus need PORT; sail is refused in ENCOUNTER.

1. Blank line: parse gives verb "" and dispatch returns "".
2. Unknown verb: isVerb is false, so dispatch prints "I don't understand 'sial'. Type help."
3. Missing argument: dispatch sees an empty argument and prints that verb's usage line.
4. Extra words: find falls back to the first word and amounts read getWord(0), so sail dover now please echoes "sail dover" and proceeds. 5, 6, 25. Case, spacing, quotes, punctuation, invisible characters: parse strips, lower-cases, splits on \s+, trims "' . , ! ? from each word's ends.
5. A real stop that is not adjacent: find succeeds, isNextTo is false, so dispatch lists the neighbours.
6. sail help: the verb is the first word only, find("help") gives null, and the reply is "no such place". 9, 11. A negative amount, "ten" or "10.1": parseCount throws NumberFormatException and dispatch asks for a whole number of 0 or more.
7. Zero: parseCount gives 0, the purchase returns 0, the reply is "bought 0".
8. Overflow: PlayerShip divides gold by price before multiplying, so hire 2147483647 buys what 200 gold covers.
9. More than the player can afford or hold: purchases cap by gold, maxHull and MAX_COUNT and report the units bought, the one case that changes state. 14, 15. A verb in the wrong mode: the dispatch mode check prints "nothing to fight" or "you must be in port".
10. An unknown item: dispatch sees a word outside cannons, armour and rum and lists those three.
11. retire away from home or short of gold: dispatch checks the stop against home and the gold against GOLD_TARGET, then prints the gold still needed.
12. The quit confirmation: dispatch reads one more line and ends the run only on "y" or "yes". 19, 20. Spamming look or status, and sail <current stop>: those branches never call roll or drink, and a target equal to the current stop gets

"you are already there".

13. Repeated flee: every flee applies FLEE_DAMAGE, so the hull drains until isDefeated ends the run as a sinking.
14. An upgrade at the maximum: the upgrade methods check the level before the gold, so the gold is untouched. 23, 27. A 100,000-character line, and injection: isVerb looks the first word up in the fixed verb table, and find and buy compare against fixed keys and three item words, so typed text is only ever data.
15. Input ends: run checks hasNextLine() before every read, prints "input ended" and sets running to false.
16. Accents and emoji: find gives null and no item word matches, giving the case 8 or case 16 message.

6. Test plan

One student. TestCase class per production class, in the default package so tests reach the protected setters. FixedRandom is a test-only Random whose nextInt returns a set value: 0 forces an encounter, the first-listed type and a zero damage roll; 99 forces none. A fresh player is a new PlayerShip at Sark. Parameterless methods get an edge state in place of bad input, getters are covered by their constructor's test, and bracketed numbers are DESIGN.md §4 cases, all 27 of which appear.

Game:

- Game(...) and run: a fresh game reports mode PORT at Sark, and the script look, status, quit, yes prints "Sark" and "Hull 100/100" and ends at OVER. A null map throws IAE; a blank line, sail, then end of input gives the unknown-command text and "input ended" with no exception [1, 2, 24].
- dispatch, normal: sail barfleur sark reaches the lane with rum 28; hire 5 gives crew 25 and gold 125; with FixedRandom(0), fight at the lane ends with gold 350 and no encounter; retire on 1000 gold ends at OVER; look, status and help print the stop name, "Morale 70" and every usage line. Bad: bare sail gives the usage line [3]; sail dover names Barfleur-Sark and West Channel [7]; sail sark says already there [20]; hire 2147483647 buys 13 men for 195 gold [12, 13]; hire at sea demands a port [15]; buy spyglass lists the three items [16]; buy cannons at level 5 keeps gold 200 [22]; fight in port says nothing to fight [14]; flee after setHull(5) sinks the ship [21]; retire on 400 gold gets "600 more needed" [17]; yse keeps the run alive [18]; three looks at sea leave the encounter null [19]; help sail ignores the extra word [4].

CommandParser and Command:

- parse, isVerb, usageOf, allUsages and parseCount: SAIL "Dover". gives "sail" and "dover" [5, 6, 25]; usageOf("sail") gives "sail" and allUsages() holds all twelve; "25" gives 25 and "0" gives 0 [10]. Tabs and spaces give verb "" [1]; sail and System.exit(0) fail isVerb [2, 27]; "-50", "ten", "10.1" and "9999999999" throw NumberFormatException [9, 11].
- Command(...), getWord and toString: from ("buy", "rum 20"), getWord(1) gives "20" and toString gives "buy rum 20"; a null verb throws IAE, getWord(5) gives "".

ChannelMap, Location and Port:

- standard, add, connect and find: 18 stops, home sark, every link both ways; dover boulogne finds the lane and dover now please finds Dover [4]. A duplicate key and connect("dover", "atlantis") throw IAE; "atlantis", "help" and "dövr" give null [8, 26].
- Location(...), connect, isNextTo, getNeighbours and describe: a lane built with chance 35 reports it; after a.connect(b), a.isNextTo(b) is true; Sark lists Barfleur-Sark then West Channel and names both in its description. a.connect(a) throws IAE, a pair connected twice lists each once, and add on the returned list throws UnsupportedOperationException.
- Port(...), getNation, rumPrice, upgradePrice and describe: Dover reports "English", chance 0, rum 4 and upgradePrice(2) of 200, and its description holds the prices; Sark reports rum 2.

Ship, PlayerShip and EnemyShip:

- Ship(...), attackStrength and isDefeated: a test subclass with maxHull 40 starts at hull 40, and a fresh player attacks for 12 undefeated. maxHull 0 and a null crew throw IAE, a crew at morale 0 attacks for 7, and setHull(0) reports defeated.
- takeDamage, addGold, spendGold, addRum and takeRum: a fresh player taking 12 loses 10 hull, leaving hull 90, crew 18 and morale 60; 50 gold added or spent gives 250 or 150; addRum(10) gives 40; takeRum(5) gives 5. Negatives throw IAE, takeDamage(500) returns 100 leaving hull 0, spendGold(500) throws IAE with gold still 200, and takeRum(50) gives 30 and empties the hold.
- PlayerShip(...), moveTo, addNotoriety and describe: a new ship sits at Sark with notoriety 0; sailing to the lane and back counts 1 port visited; addNotoriety(2) gives 2; describe holds every field. A null home, a move from Sark to Dover and a negative notoriety throw IAE.
- repair, hire, buyRum, upgradeCannons, upgradeArmour and payBonus: from setHull(80), repair(10, 2) returns 10 leaving gold 180; hire(5, 15) returns 5; buyRum(20, 2) returns 20; an upgrade returns true at level 2 leaving gold 0; payBonus(100) after onWin(600) gives greed 25 and morale 82. repair(50, 2) at hull 80 returns 20 [13], hire(30, 15) returns 13 leaving gold 5 [12], buyRum(20, 4) with 3 gold returns 0, an upgrade at level 5 or on 50 gold returns false [22], and payBonus(500) returns false.
- EnemyShip(...), describe and the EnemyType getters: (MERCHANT, 0) gives hull 40, crew 8, gold 150 and morale 30, and MERCHANT.getGain() gives 1. (PIRATE, -1) and (null, 0) throw IAE, and COAST_GUARD.getGain() gives 3.

Crew and Encounter:

- `Crew(...)`, `lose`, `hire`, `moraleFactor` and `mutinied`: `(20, 70)` gives factor 0.85 and has not mutinied; `lose(2)` returns 2 leaving count 18 and morale 60; `hire(5)` returns 5. `(50, 70)` caps at 40, `lose(25)` returns 20 leaving morale 0 and a mutiny, `hire(30)` returns 20 [13], and negatives throw IAE.
- `drink`, `onWin`, `onFlee` and `receiveBonus`: with 30 bottles the crew of 20 drinks 2 holding morale 70; `onWin(150)` gives morale 80 and greed 7; `onFlee` gives 55; a 100 gold bonus after `onWin(600)` gives greed 25 and morale 82. `drink(1)` drinks 1 and drops morale to 65 for the one bottle short, `drink(0)` drops it to 60, `onWin(5000)` caps greed at 100, and negatives throw IAE.
- `Encounter(...)`, `roll`, `describe`, `fight` and `flee`: a sea lane with `FixedRandom(0)` gives a merchant encounter described as a merchant; fighting it takes the surrender in round 3, ending at hull 97, gold 350, rum 38, morale 80 and greed 7; fleeing gives hull 92, crew 19 and morale 50. A null enemy throws IAE; a port, or the high seas with 99, rolls null; after `setHull(5)` the same fight against `(PIRATE, 0)` loses in round 1 with gold still 200, and fleeing empties the hull [21].

7. Division of work

Aidan owns `CommandParser`, `Command`, `Ship`, `PlayerShip`, `EnemyShip`, `EnemyType`, `Crew` and `Encounter`, with cases 1–6, 8–13, 18 and 21–27, and the Web-CAT submission. Marcos owns `Game`, `ChannelMap`, `Location`, `Port` and `Describable`, with the map, all *default* numbers, cases 7, 14–17, 19 and 20, and the play-through transcript for the oral defence.

Day one, together: every class written with its section 4 signatures and stub bodies, so the project compiles and each of us edits only our own files. Week 1 milestone, at the Tuesday 7 pm meeting: start at Sark and run `look`, `status`, `sail` round all 18 stops, `repair`, `hire`, `buy`, `help` and `quit`. Week 2 adds fights, the crew rules, bonus, retire, mutiny and sinking, and we both play to set the numbers. Marcos owns `Game`, the only class calling both sides' code, so nobody edits the same file; each class lands on `main` with its test class, and the merged build must pass Web-CAT with no style warnings.

If time runs short we cut, each cut leaving every other class unchanged: greed and bonus first; then rum, with a fixed morale drop per sea move; then the three high-seas stops.

8. Revised scope

Each item gives the change, the reason, and where it came from.

1. An 18-stop metro map of ports, sea lanes and high seas, each kind with its own encounter chance, `sail` moving one stop: the route chosen changes the risk and needs no route-finding. Marcos; group discussion dropped a 25-stop draft's coastal waters so a crossing takes two moves.
2. A `HashMap` from key to stop with an `ArrayList` of neighbours per stop, closing the §5 unknown about coding the map. Aidan's §5 idea and `HashMap` choice.
3. Crew morale, greed and rum moved from stretch goal 3 into the MVP in basic form (DESIGN.md FR15–17) with bonus added, so morale drives combat for both sides and fights gain a second ending. Marcos; the basic and full split from GenAI review.
4. An abstract `Ship` shared by both sides, with enemies scaled by `EnemyType` and notoriety: one fight routine, and the difficulty scaling promised in DESIGN.md §1 returns. Marcos asked for a shared base; the GenAI draft chose inheritance.
5. Cannon and armour levels on one price list in place of weapon items with per-port stock, removing an `Item` class and seven stock lists. GenAI review, after Marcos set a two-week budget.
6. Overflow (case 12) handled by dividing before multiplying, refusals as return values with `dispatch` writing every message, and a `parse` that never throws, so one place makes all player-facing text. Aidan's draft; the division from GenAI review.
7. The mode computed by `Game.mode()` with `OVER` as one value, closing the §5 unknown about restricting commands by mode. Aidan's mode idea plus DESIGN.md's "state machine" note.
8. One `Random` injected through `Game`, with `Encounter` fully constructible, closing the §5 unknown about forcing a fight in `JUnit`. GenAI review of §5 in both drafts.
9. No enums: verbs, modes, kinds and nations are `String` constants with a `HashMap` verb table, and `EnemyType` is a class with three static `final` instances, because the course has covered `HashMap`, interfaces and generics but not enums. Marcos, after GenAI review proposed enums.